

Stability of Self-adaptive Software Architectures

Maria Salama

School of Computer Science
The University of Birmingham
Birmingham B15 2TT, UK
Email: m.salama@cs.bham.ac.uk

Abstract—Stakeholders and organisations are increasingly looking for long-lived software. As architectures have a profound effect on the operational life-time of the software and the quality of the service provision, architectural stability could be considered a primary criterion towards achieving the long-livety of the software. Architectural stability is envisioned as the next step in quality attributes, combining many inter-related qualities. This research suggests the notion of *behavioural stability* as a primary criterion for evaluating whether the architecture maintains achieving the expected quality attributes, maintaining architecture robustness, and evaluating how well the architecture accommodates run-time evolutionary changes. The research investigates the notion of architecture stability at run-time in the context of self-adaptive software architectures. We expect to define, characterise and analyse this intuitive concept, as well as identify the consequent trade-offs to be dynamically managed and enhance the self-adaptation process for a long-lived software.

I. MOTIVATION

Modern software systems are increasingly becoming complex, heterogeneous, pervasive, and tend to operate in unpredictable environments. This growth, which is likely to continue into the foreseeable future, has motivated the need for long-lived software, as the cost of building these systems is becoming high. The uncertainty arising from the continuous changing requirements during run-time and the likelihood of changes adds another dimension to the challenge. Such challenges can have impact on the software life-time and the quality of the service provided. Also, evolution is becoming more critical for modern software systems [1]. The emergence of new software paradigms; such as mobile, and cloud; has increasingly made the need to transit to these new paradigms.

An essential prerequisite for long-lived software is its capability to maintain services provision with expected qualities and continue to accommodate changes in requirements and environment. This imposes new questions on how to design, operate, maintain and cost-effectively evolve such systems. In particular, architects and practitioners are challenged by how they can systematically design architectures that are flexible yet dependable in supporting likely changes in requirements and the environment. An increasing number of architectures and approaches are being built to accommodate evolution.

As architectures have a profound effect on the life-span of the software and the quality of the service provision, the stability of the architecture tends to reflect on the success of the system in supporting changes and evolution over time. *Architectural Stability* refers to the extent to which the architecture can support changes in requirements (functional and non-functional) and the operating environment, while reducing the likelihood of architectural drift and phasing-out [2]. Stability

can be deemed to be of strategic importance and can reflect on the architectural performance in meeting change, unlocking potentials, and rendering strategic benefits. Examples include the opportunity to instantiate from the architecture new market products, the flexibility to respond to competitive forces and changing market conditions, and the ability to accommodate new services. Stability may also render long-term operational benefits; e.g. reduced maintenance cost.

II. LITERATURE REVIEW

Architectural stability was defined in the literature from different perspectives. This term was first introduced as a concept reflecting the architecture structure intactness that is evaluated during design-time [3]. Stability was also defined from the evolution perspective [4] [5] [6], since the concept refers to the ability of accommodating changes over the time. Reviewing the state-of-art of the software engineering literature [7] attests that architectural stability was implicitly considered and inter-linked with other quality attributes; such as resilience, robustness and trustworthiness. Also, stability was found measured and evaluated implicitly within the approaches and mechanisms of designing and evaluating architectures, with the exception of few efforts [8] [4] [9]. Though the concept appeared to be prominent among the different software engineering practices, this consideration was not explicit and further efforts are needed for explicitly addressing, measuring, evaluating and achieving stability.

Considering modern software systems complexity and architecture evolution bring in concerns a wider view for the concept that should include the run-time dynamism of modern software systems and their environment uncertainty. This indicates a shift from a narrow concept of the stability (architecture intactness) to a wider concept that tackles dynamism, uncertainty and evolution faced by architectures. In the context of self-adaptive software architectures, the stability state defined at design-time could not endure as a stability state while continuously adapting in response to the changing requirements. Thus, architectural stability has to consider a dynamic view of the world, that will consider not only the stability of the architecture structure during design-time, but also the stability of the architecture behaviour during run-time.

From our earlier work reviewing the state-of-art related to architecture stability [7], we have drawn out dimensions related to *time* (design/run-time) and *aspect* of stability (architecture structure/function/behaviour), for defining and characterising the architectural stability, to provide more holistic and comprehensive understanding of this notion and related problems. We argue that a global and general definition for stability might not be accurate. The definition of stability shall be considered

relative to the case under question. A sensible treatment for stability can combine the *when* and *what* dimensions. For instance, we can define *design-time structural stability* as the extent to which the architecture structural design decisions will retain intact in the face of future changes and operate in unpredictable and uncertain environments, and *run-time behavioural stability* as the ability to fulfil the changing quality requirements during run-time.

Modern and complex software systems necessitate such comprehensive view of stability, considering the different aspects of stability (structural, functional, and behavioural stability) while designing architectures, as well as during their operation. During the stage of designing the architecture, a candidate architecture should be evaluated by the ability of its structure (structural stability) to maintain fulfilling the functional requirements (functional stability) and quality requirements (behavioural stability) that are known at this stage, as well as the likely changes to occur in the future. When the software is put into operation, the architecture ability to fulfil the changing behavioural requirements and workloads (behavioural stability) should be continuously assessed during run-time. In case the architecture structure fails to fulfil the functional or non-functional requirements, or the functional requirements have widely changed, re-designing the architecture or making ripple modifications are required to take place offline before putting the architecture back into operation. Thus, one can consider structural and functional stability are implicitly embedded at run-time stability. While design-time stability was explicitly tackled in the literature, there is no generic framework, to the best of the researcher knowledge, for analysing and evaluating the stability of software architectures during run-time, accommodating different nuances of this kind of stability.

III. RESEARCH PROBLEM AND OBJECTIVES

A. Problem Formulation

In the run-time evolutionary context of software architectures, there is a pressing need for performing the run-time adaptation in a way that the architecture is left robust on the long-term, fulfilling not only the current run-time quality requirements, but also the uncertain changing future requirements, for longer-living software. This research suggests the notion of *run-time behavioural stability* as a primary criterion for maintaining the architecture success in accommodating changes in quality requirements on the long-term under uncertainty. The architecture stability is meant to be the ability to continue delivering the expected quality and environmental run-time requirements, while the architecture is left robust on the long-term. We argue that the architecture stability should be seen in the context of run-time; i.e., architectural stability is a quality that should be continuously considered, achieved and monitored during run-time while adaptation is taking place to meet the changing run-time quality requirements.

Problem. *Self-adaptive software needs a systematic way to analyse, measure and assess the Architectural Stability under the run-time uncertainty, to maintain the architecture robust for a long-lived software.*

Research Claim. *Architectural Behavioural Stability is a quality attribute that needs to be continuously maintained during run-time for self-adaptive software architectures. This*

attribute can be analysed, measured and managed, in order support the self-adaptation process for the long-term goal of having a robust architecture facing uncertain changing run-time quality requirements.

The proposed notion will adopt a more proactive and dynamic attitude towards architectural stability. Intuitively, the proposed notion embeds the enhancement of the self-adaptation process that will be performed according to long-term goals and accommodating multiple trade-offs, as architectures that exhibit a good trade-off between multiple quality attributes is highly desirable for self-adaptive software. The relevance of the research increases with the uncertainty of the requirements changing during run-time. Our research focuses on the architectural level adaptation for two reasons: (1) the foundation for any software is its architecture, which allows or precludes nearly all of the quality attributes; (2) software architecture describes its high level structure and behaviour, thus, it exposes the dimensions along which a software is expected to evolve and provides basis for evolution.

The research main question is the notion of *run-time behavioural stability* of self-adaptive software architectures; i.e. how can we systematically analyse and evaluate the stability of self-adaptive and self-managed software architectures during run-time. More specifically, the research questions are formulated as follows: (1) *How to analyse architectural stability for self-adaptive software and what are the primitives for run-time consideration?* (2) *How to model, measure and evaluate the run-time behavioural stability of self-adaptive architectures?* (3) *How to systematically reason and manage trade-offs between quality requirements at run-time?*

B. Research Objectives

The research aims to provide a systematic framework to support the process of reasoning and characterising the stability of software architectures during run-time under uncertainty. The research attempts to capture the essence and nuances of run-time behavioural stability; by characterising this intuitive idea, and analysing its primitives. The research intends to set the grounds for this new era, calling for a novel approach for evaluating the stability of the architecture in self-adaptive architectures through dynamic measurement, analysis, control and feedback. The benefits of such evaluation are: (1) assisting in software adaptation under uncertainty, (2) continuously meeting behavioural requirements, (3) reducing long-term maintenance cost, (4) engineering for long-lived architectures, and (5) minimising ramifications of architectural erosion. Particularly, the run-time yield is enhancing the adaptation process, by informing it during run-time for the choice of adaptation strategies that keeps the architecture stable on the long-term, while the long-term yield is having long-lived software, which keeps meeting functional, non-functional and environmental requirements.

IV. PROPOSED SOLUTION

The proposed solution is a systematic framework to support the process of characterising, reasoning and managing the stability of architectures at run-time. Generally, the solution is an approach to enhance the run-time adaptation. The approach will intend to analyse the primitives of architecture stability, model the run-time goals, symbiotically related them

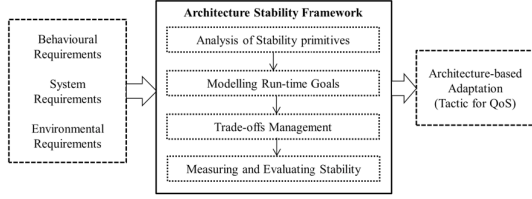


Fig. 1. Stability Framework

to architectures, and manage the trade-offs between quality requirements. The output of the trade-offs analysis will inform the choice of an adaptation strategy that will, on the long-term, keep the architecture robust. By adaptation strategy, we limit our work on architectural tactics that are designed to fulfil quality attributes. An architectural tactic is a means of satisfying a quality-attribute-response measure by manipulating some aspects of a quality attribute model through architectural design decisions [10].

For characterising the concept of architectural stability, we tend to formalise a prototype implementation framework (as depicted in Fig. 1) to define, measure and evaluate the architecture stability. The tentative approach towards solving the imposed research questions undertakes the following (details are given in the following sub-sections):

- 1) Analysing Architectural Stability.
- 2) Modelling run-time goals.
- 3) Managing run-time architecture trade-offs.
- 4) Measuring and evaluating Architectural Stability.

A. Analysing Architectural Stability

We propose a taxonomy that describes the various facets of architectural stability. The underlying insight guiding our proposal is that well-understood quality attributes; such as performance, and dependability; has a well-established taxonomies in the literature. This aims to give precise characterisation and better understanding for the various concepts that come into play when addressing the architecture stability. The taxonomy would serve for characterising, analysing, achieving and evaluating stability as an architectural property.

When addressing stability as an architectural property, we propose characterising it by the quality attributes that are essential to analyse it, understand it, and achieve it. We also need to specify the means to realise it throughout the software life-cycle; since the architecture tends to interleave and intertwine with other engineering phases; including requirements, architecture design and evaluation, maintenance and evolution. Also, the threats that may affect the stability of the architecture during its entire life should be identified in order to be systematically handled and managed. Thus, we propose the taxonomy of architectural stability to embody: (i) attributes, (ii) threats, and (iii) means.

B. Modelling run-time goals

This includes having fine-grained knowledge representation of the behavioural/quality requirements (run-time goals). The run-time goals could be generally categorised as:

- *Behavioural requirements*: including workloads, quality requirements, and environmental constraints.

- *System requirements*: representing the limited computational resources and operational primitives.
- *Environmental requirements*: including energy consumption and environmental constraints, to achieve the greenability quality attribute and sustainability on the long-term.

The run-time goals modelling allows realising the symbiotic relation between these goals and the architecture, promising more accurate adaptation decisions and better up-to-date/informed management for trade-offs. The symbiotic relation aims to keep the run-time goal model live and up-to-date reflecting on the extent to which adaptation decisions satisfies the goal(s).

C. Modelling and managing run-time trade-offs

Since the architecture is expected to encounter trade-offs when fulfilling multiple run-time quality goals, trade-offs management process is required in such cases when selecting the adaptation strategy. We draw on game theory when selecting the adaptation strategy to take a strategic decision under the run-time uncertainty while managing the run-time trade-offs. In more details, we tend to evaluate adaptation strategies for their pay-off values, and based on such evaluation, an adaptation strategy will be selected in a way to support trade-offs management between different quality requirements under uncertainty and environmental constraints.

D. Measuring and evaluating Architectural Stability

The measurement and evaluation of stability is two-folds: (i) measuring the effect of applying an adaptation strategy on quality attributes to ensure achieving the run-time goals, and (ii) evaluating the architecture stability using probabilistic model-checking. The former includes architecture-level dynamic QoS modelling and evaluation of the effect and performance of adaptation tactics with respect to quality targets. The latter includes defining an approach for the verification of self-adaptive architectures that relies on historical data and probabilistic model-checking for assessing the architecture stability. The historical data collected is aggregated into a probabilistic model of the systems behaviour that will be used as input into a model checker for evaluating whether quality properties are satisfied. This evaluation focuses on whether a set of quality attributes are satisfied during operation, which enables assessing whether the system is able to maintain trustworthy service delivery on the long-term, in spite of changes in run-time requirements and workloads, as well as uncertainty of the operating environment.

V. EXPECTED CONTRIBUTIONS AND EVALUATION PLAN

A. Novelty and Expected Contributions

To the best of the researcher knowledge, there is no generic framework for analysing the stability of software architectures during run-time. The novelty of this research relies on the systematic characterising and modelling of stability during run-time. The use of Game Theory in managing trade-offs arising under the run-time uncertainty is another novel dimension. The expected novel contributions of the thesis are: (1) a taxonomy for characterising and describing the various facets of architectural stability as a quality attribute, capturing the essence of run-time architectural stability, outlining its metrics,

as well as characterising its primitives, and (2) a sound method and prototype implementation framework to analyse and evaluate stability during run-time, as well as manage the consequent trade-offs.

B. Evaluation Plan

Though the fundamentals of the research could work with the various clusters of self-adaptive software, this research will focus on self-awareness as a promising architectural concept recently emerging in the field of self-adaptive software [11]. Inspired from psychology and cognitive science, the concept of self-awareness has been re-deduced in the context of software engineering to realise software autonomic behaviour [11]. Different levels of self-awareness, called capabilities, have been identified: stimulus-, interaction-, time-, goal-, and meta-self-awareness [11]. The self-aware architecture style, defined by [11], draws on the principles of self-awareness to enrich the capabilities of self-adaptive architectures. The case of self-aware architecture pattern represents a challenge for evaluating the work, as such architectures exhibit complex trade-offs across multiple dimensions - goals, time, and interaction - emerging internally and externally from the uncertainty of the operation environment.

We demonstrate the instantiation of our proposed framework to the case of cloud auto-scaling architecture, in which the temporal goal-sharing self-aware architecture pattern is used [12]. Auto-scaling is used as its architecture is essentially quality-driven and -optimising. The choice of this case is attributed to the following: (i) QoS-sensitivity; QoS tends to fluctuate based on sharing, dynamic use, (ii) elasticity in the run-time demand; architecture is expected to adapt to meet the changes in run-time demand, and (iii) the selected pattern features all level of self-awareness, which makes it possible to encounter multiple trade-offs with the presence of different levels of awareness.

We plan to use *simulation and dynamic analysis*, as controlled experimental validation method, to evaluate the accuracy, expressiveness and adequacy of the proposed framework reasoning about stability. The efficiency of the selected adaptation strategy will, then, be measured on consequent time instances compared with the output of the model-checker, for achieving the targeted stability. More specifically, we plan to:

- Verify the accuracy of the prediction by applying the conceptual model to consequent time instances during run-time.
- Analyse the systems ability to deal with uncertainty by comparing the results of consecutive run-time instances and checking the ability of fulfilling the changing requirements.
- Apply the framework to an extra-growing auto-scaling architecture, for stress-testing the efficiency of the model.

VI. RESEARCH PLAN AND PROGRESS

The plan for this research can be summarised as follows: (1) conducting the literature review, (2) developing the stability taxonomy and analysing its primitives, (3) modelling the run-time goals, (4) developing the approach for managing trade-offs, (5) evaluating the stability by developing the probabilistic

model-checking, and (6) conducting the evaluation for the thesis.

This dissertation is currently in an initial stage. The motivation and problem statement have been defined. The overall conceptual framework is defined as described in this paper. A thorough structured literature review was conducted to create a solid basis for this work. The literature review conducted is in two-fold: (1) state of art about architectural stability, and (2) systematic mapping study about managing architectural trade-offs. Both have been submitted for publications and are currently under review. The task of developing the stability taxonomy has been accomplished, and is to be submitted for publication. The task of modelling run-time goals is currently under progress. As self-aware architectures have been recently introduced, the task of embedding quality-driven tactics to self-aware architectures was required prior to the trade-offs management. This task has been accomplished and accepted for publication in the IEEE CLOUD conference. As a next step in the context of this thesis, the trade-offs management approach will be developed, followed by the stability evaluation tasks.

ACKNOWLEDGMENT

Maria Salama is funded under a studentship from The University of Birmingham. The author would like to thank Rami Bahsoon for inspiring discussions and feedback.

REFERENCES

- [1] J. Buckley, T. Mens, M. Zenger, A. Rashid, and G. Kniesel, "Towards a taxonomy of software change," *Journal of Software Maintenance and Evolution: Research and Practice*, vol. 17, no. 5, pp. 309–332, 2005.
- [2] K. H. Bennett and V. T. Rajlich, "Software maintenance and evolution: a roadmap," ACM, 2000, Conference Paper, pp. 73–87.
- [3] R. Bahsoon and W. Emmerich, "Evaluating software architectures: development, stability, and evolution," in *ACS/IEEE International Conference on Computer Systems and Applications, 2003. Book of Abstracts.*, 2003, Conference Proceedings, p. 47.
- [4] M. Jazayeri, "On architectural stability and evolution." Springer-Verlag, 2002, Conference Paper, pp. 13–23.
- [5] D. Kelly, "A study of design characteristics in evolving software using stability as a criterion," *IEEE Transactions on Software Engineering*, vol. 32, no. 5, pp. 315–329, 2006.
- [6] A. L. Medeiros, E. Figueiredo, I. Galvao, A. Garcia, T. Batista, and C. Sant'Anna, "Concern-based assessment of architectural stability: A comparative study," 27-29 Sept. 2010 2010, Conference Paper, pp. 130–139.
- [7] M. Salama and R. Bahsoon, "Architecture stability: Survey of the state of art, challenges and future directions," *ACM Computing Surveys (CSUR)*, 2015, under review.
- [8] R. Bahsoon and W. Emmerich, "Evaluating architectural stability with real options theory," 11-14 Sept. 2004 2004, Conference Paper, pp. 443–447.
- [9] A. Gorbenko, V. Kharchenko, O. Tarasyuk, Y. Chen, and A. Romanovsky, "The threat of uncertainty in service-oriented architecture," ACM, 2008, Conference Paper, pp. 49–54.
- [10] F. Bachmann, L. Bass, and M. Klein, "Deriving architectural tactics: A step toward methodical architectural design," Software Engineering Institute, Carnegie Mellon University, Report Technical Report CMU/SEI-2003-TR-004, 2003. [Online]. Available: <http://resources.sei.cmu.edu/library/asset-view.cfm?AssetID=6593>
- [11] F. Faniyi, P. Lewis, R. Bahsoon, and Y. Xin, "Architecting self-aware software systems," April 2014 2014, Conference Paper, pp. 91–94.
- [12] T. Chen, F. Faniyi, R. Bahsoon, P. Lewis, X. Yao, L. Minku, and L. Esterle, "The handbook of engineering self-aware and self-expressive systems," Report Technical Report, 2014.